FINAL CODE OF FIRE FIGHTER ROBOT 🤖🔥

```
/*
  FIRE FIGHTING + GSM ALERT ROVER - ARDUINO UNO MEMORY OPTIMIZED
  Original behavior preserved. Constant serial/GSM strings are stored in FLASH.
*/

#include <AFMotor.h>
#include <Servo.h>
#include <SoftwareSerial.h>
#include <avr/pgmspace.h>


// ================================================================
//                    MOTOR OBJECTS
// ================================================================

AF_DCMotor M1(1);   // LEFT FRONT
AF_DCMotor M2(2);   // LEFT REAR
AF_DCMotor M3(3);   // RIGHT FRONT
AF_DCMotor M4(4);   // RIGHT REAR


// ================================================================
//                    SENSOR PINS
// ================================================================
```

```
#define FLAME_LEFT     A0
#define FLAME_FRONT    A1
#define FLAME_RIGHT    A2

#define GAS_SENSOR     A3


// ================================================================
//                  SIM800L
// ================================================================

#define SIM800_RX      A4
#define SIM800_TX      2

SoftwareSerial sim800L(SIM800_RX, SIM800_TX);


// ================================================================
//                  OUTPUTS
// ================================================================

#define PUMP_RELAY     A5
#define SERVO_PIN      9
```

```
// ============================================================
//                  SERVO
// ============================================================

Servo nozzleServo;

#define SERVO_LEFT     50
#define SERVO_CENTER    90
#define SERVO_RIGHT     130


// ============================================================
//                  PHONE
// ============================================================

const char PHONE_NUMBER[] = "+919037790678";


// ============================================================
//                  MOTOR SPEED
// ============================================================

#define MOTOR_SPEED       180
#define TURN_SPEED        160
#define FINE_ALIGN_SPEED   130
```

```
// ============================================================
//              ALIGNMENT SETTINGS
// ============================================================

#define ALIGN_STEP_TIME    90
#define ALIGN_PAUSE       35
#define FINE_ALIGN_TIME    55

#define MAX_ALIGN_STEPS    80
#define MOVE_FORWARD_BEFORE_SPRAY  400


// ============================================================
//             FIRE CONFIRMATION
// ============================================================

#define FIRE_CONFIRM_COUNT  2
#define FIRE_CONFIRM_DELAY  30


// ============================================================
//             EXTINGUISH SETTINGS
// ============================================================

#define MAX_EXTINGUISH_CYCLES  5
```

```
#define SERVO_SETTLE_TIME     250
#define PUMP_START_DELAY      250

#define SWEEP_STEP_DELAY      25

#define RECHECK_DELAY         300



// ================================================================
//                FIRE CONTROLLER STATES
// ================================================================

enum FireMode
{
   FIRE_IDLE,

   FIRE_WAIT_FOR_CALL,

   FIRE_ALIGN,

   FIRE_SPRAY_SETTLE,

   FIRE_SPRAY_LEFT,

   FIRE_SPRAY_RIGHT,
```

```
  FIRE_RECHECK
};

FireMode fireMode = FIRE_IDLE;

// ================================================================
//                FIRE VARIABLES
// ================================================================

bool fireActive = false;

unsigned long fireTimer = 0;

int extinguishCycle = 0;

int alignStep = 0;

int currentServoAngle = SERVO_CENTER;

// Call-first fire alert sequence.
// These are fire-controller variables, not GSM logic.
bool fireCallStarted = false;
bool fireCallWaiting = false;
```

```
bool fireGasDetected = false;

// ================================================================
// FIRE EVENT LATCH
// Once a fire is confirmed, no new GSM call/SMS event can be
// started until the fire has been confirmed extinguished.
// ================================================================

bool fireEventLatched = false;

int fireClearConfirmCount = 0;

#define FIRE_CLEAR_CONFIRM_COUNT 5



// ================================================================
//                FIRE SENSOR STRUCTURE
// ================================================================

struct FireState
{
  bool left;
  bool front;
  bool right;
  bool detected;
};
```

```
// ================================================================
//                    GAS ALERT
// ================================================================

bool gasAlertSent = false;



// ================================================================
//                  GSM STATES
// ================================================================

enum GSMState
{
  GSM_IDLE,

  GSM_SMS_CMGF_WAIT,
  GSM_SMS_CMGS_WAIT,
  GSM_SMS_BODY_WAIT,

  GSM_CALL_DIAL_WAIT,
  GSM_CALL_ACTIVE
};

GSMState gsmState = GSM_IDLE;
```

```
// ================================================================
//                GSM VARIABLES
// ================================================================

unsigned long gsmTimer = 0;
unsigned long callStartTime = 0;

char pendingSMS[180];

bool smsPending = false;
bool callPending = false;

// ================================================================
//             CLEAR SIM800 BUFFER
// ================================================================

void clearSIM800()
{
  while (sim800L.available())
  {
    sim800L.read();
  }
}

// ================================================================
```

```
//            PROCESS SIM800 INPUT
// ================================================================


void processSIM800Input()
{
  while (sim800L.available())
  {
    char c = sim800L.read();


    Serial.write(c);
  }
}


// ================================================================
//                REQUEST SMS
// ================================================================


bool requestSMS(const char *message)
{
  if (gsmState != GSM_IDLE)
  {
    return false;
  }


  strncpy(
    pendingSMS,
```

```
      message,
      sizeof(pendingSMS) - 1
    );

    pendingSMS[
      sizeof(pendingSMS) - 1
    ] = '\0';

    smsPending = true;

    return true;
}

// ============================================================
//                  REQUEST CALL
// ============================================================

bool requestCall()
{
    callPending = true;

    return true;
}

// ============================================================
//                  START SMS
```

```
// ================================================================


void startSMS()
{
   Serial.println();
   Serial.println("--------------------------------");
   Serial.println("       GSM: START SMS");
   Serial.println("--------------------------------");

   clearSIM800();

   sim800L.println("AT+CMGF=1");

   gsmTimer = millis();

   gsmState = GSM_SMS_CMGF_WAIT;

   smsPending = false;
}

// ================================================================
//                  START CALL
// ================================================================

void startCall()
{
```

```
  Serial.println();
  Serial.println("--------------------------------");
  Serial.println("       GSM: START CALL");
  Serial.println("--------------------------------");

  clearSIM800();

  sim800L.print("ATD");
  sim800L.print(PHONE_NUMBER);
  sim800L.println(";");

  gsmTimer = millis();

  gsmState = GSM_CALL_DIAL_WAIT;

  callPending = false;
}

// ================================================================
//                  GSM UPDATE
// ================================================================

void updateGSM()
{
  processSIM800Input();
```

```
unsigned long now = millis();

// ============================================================
// IDLE
// ============================================================

if (gsmState == GSM_IDLE)
{
  if (smsPending)
  {
    startSMS();
    return;
  }

  if (callPending)
  {
    startCall();
    return;
  }

  return;
}

// ============================================================
// SMS: CMGF
// ============================================================
```

```
if (gsmState == GSM_SMS_CMGF_WAIT)
{
  if (now - gsmTimer >= 1000)
  {
    sim800L.print("AT+CMGS=\"");
    sim800L.print(PHONE_NUMBER);
    sim800L.println("\"");

    gsmTimer = now;

    gsmState = GSM_SMS_CMGS_WAIT;
  }

  return;
}

// ============================================================
// SMS: CMGS
// ============================================================

if (gsmState == GSM_SMS_CMGS_WAIT)
{
  if (now - gsmTimer >= 1500)
  {
    sim800L.print(pendingSMS);
```

```
      sim800L.write(26);   // CTRL+Z

      gsmTimer = now;

      gsmState = GSM_SMS_BODY_WAIT;
    }

    return;
}

// ============================================================
// SMS: COMPLETE
// ============================================================

if (gsmState == GSM_SMS_BODY_WAIT)
{
    if (now - gsmTimer >= 5000)
    {
      Serial.println();
      Serial.println("GSM: SMS COMPLETE");

      gsmState = GSM_IDLE;
    }

    return;
```

```
}

// ============================================================
// CALL: DIALING
// ============================================================

if (gsmState == GSM_CALL_DIAL_WAIT)
{
  if (now - gsmTimer >= 2000)
  {
    Serial.println("GSM: CALL ACTIVE");

    callStartTime = now;

    gsmState = GSM_CALL_ACTIVE;
  }

  return;
}

// ============================================================
// CALL: ACTIVE
// ============================================================

if (gsmState == GSM_CALL_ACTIVE)
{
```

```
      if (now - callStartTime >= 10000)
      {
        sim800L.println("ATH");

        Serial.println("GSM: CALL ENDED");

        gsmState = GSM_IDLE;
      }

      return;
    }
}



// ================================================================
//              PRINT FIRE SENSOR STATUS
// ================================================================

void printFireState(FireState fire)
{
   Serial.print(F("[SENSORS] LEFT="));
   Serial.print(fire.left);

   Serial.print(F(" FRONT="));
   Serial.print(fire.front);
```

```
  Serial.print(F(" RIGHT="));
  Serial.print(fire.right);

  Serial.print(F(" FIRE="));
  Serial.println(fire.detected);
}


// ================================================================
//              PRINT FIRE MODE
// ================================================================

void printFireMode()
{
  Serial.print(F("[FIRE MODE] "));

  switch (fireMode)
  {
    case FIRE_IDLE:
      Serial.println(F("IDLE"));
      break;

    case FIRE_WAIT_FOR_CALL:
      Serial.println(F("WAITING FOR CALL"));
      break;
```

```
      case FIRE_ALIGN:
        Serial.println(F("ALIGN"));
        break;

      case FIRE_SPRAY_SETTLE:
        Serial.println(F("SPRAY SETTLE"));
        break;

      case FIRE_SPRAY_LEFT:
        Serial.println(F("SPRAY 50 -> 130"));
        break;

      case FIRE_SPRAY_RIGHT:
        Serial.println(F("SPRAY 130 -> 50"));
        break;

      case FIRE_RECHECK:
        Serial.println(F("RECHECK"));
        break;
    }
}



// ================================================================
//                PRINT SYSTEM STATUS
// ================================================================
```

```
void printSystemStatus()
{
  Serial.println();
  Serial.println(F("----------------------------------------"));
  Serial.println(F("SYSTEM STATUS"));
  Serial.println(F("----------------------------------------"));

  Serial.print(F("Fire Active : "));
  Serial.println(
    fireActive ? "YES" : "NO"
  );

  printFireMode();

  Serial.print(F("Cycle      : "));
  Serial.println(extinguishCycle);

  Serial.print(F("Servo      : "));
  Serial.println(currentServoAngle);

  Serial.print(F("Pump       : "));

  if (digitalRead(PUMP_RELAY))
    Serial.println(F("ON"));
  else
```

```
      Serial.println(F("OFF"));

   Serial.print(F("GSM State   : "));
   Serial.println(gsmState);

   Serial.println(F("----------------------------------------"));
}



// ================================================================
//                MOTOR SPEED
// ================================================================

void setMotorSpeed(int speed)
{
   speed = constrain(speed, 0, 255);

   M1.setSpeed(speed);
   M2.setSpeed(speed);
   M3.setSpeed(speed);
   M4.setSpeed(speed);
}



// ================================================================
//                  STOP MOTORS
```

```
// ================================================================

void stopMotors()
{
  M1.run(RELEASE);
  M2.run(RELEASE);
  M3.run(RELEASE);
  M4.run(RELEASE);
}


// ================================================================
//                  FORWARD
// ================================================================

void moveForward()
{
  setMotorSpeed(MOTOR_SPEED);

  M1.run(BACKWARD);
  M2.run(BACKWARD);

  M3.run(FORWARD);
  M4.run(FORWARD);

  Serial.println(F("[MOTOR] FORWARD"));
```

```
}


// ============================================================
//                   BACKWARD
// ============================================================

void moveBackward()
{
  setMotorSpeed(MOTOR_SPEED);

  M1.run(FORWARD);
  M2.run(FORWARD);

  M3.run(BACKWARD);
  M4.run(BACKWARD);

  Serial.println(F("[MOTOR] BACKWARD"));
}


// ============================================================
//                    LEFT
// ============================================================

void turnLeft()
```

```
{
  setMotorSpeed(TURN_SPEED);

  M1.run(FORWARD);
  M2.run(FORWARD);

  M3.run(FORWARD);
  M4.run(FORWARD);

  Serial.println(F("[MOTOR] LEFT"));
}



// ============================================================
//                    RIGHT
// ============================================================

void turnRight()
{
  setMotorSpeed(TURN_SPEED);

  M1.run(BACKWARD);
  M2.run(BACKWARD);

  M3.run(BACKWARD);
  M4.run(BACKWARD);
```

```
  Serial.println(F("[MOTOR] RIGHT"));
}



// ============================================================
//                READ FIRE SENSORS
// ============================================================

FireState readFireSensors()
{
  FireState fire;

  fire.left =
    (digitalRead(FLAME_LEFT) == LOW);

  fire.front =
    (digitalRead(FLAME_FRONT) == LOW);

  fire.right =
    (digitalRead(FLAME_RIGHT) == LOW);

  fire.detected =
    fire.left ||
    fire.front ||
    fire.right;
```

```
  return fire;
}


// ============================================================
//              CHECK FIRE STILL PRESENT
// ============================================================

bool fireStillDetected()
{
  FireState fire = readFireSensors();

  return fire.detected;
}


// ============================================================
//                CONFIRM FIRE
// ============================================================

bool confirmFire()
{
  Serial.println(F("[FIRE] Confirming detection..."));

  for (int i = 0;
```

```
      i < FIRE_CONFIRM_COUNT;
      i++)
  {
    FireState fire = readFireSensors();

    printFireState(fire);

    if (!fire.detected)
    {
      Serial.println(F("[FIRE] Confirmation FAILED"));

      return false;
    }

    delay(FIRE_CONFIRM_DELAY);
  }

  Serial.println(F("[FIRE] FIRE CONFIRMED"));

  return true;
}



// ============================================================
//                  GAS ALERT
// ============================================================
```

```
void handleGasAlert()
{
  if (gasAlertSent)
    return;


  if (fireActive)
    return;

  if (fireEventLatched)
    return;


  if (gsmState != GSM_IDLE)
    return;


  if (requestSMS(
    "WARNING! Smoke or hazardous gas detected. Robot is waiting for fire."
  ))
  {
    gasAlertSent = true;

    Serial.println();
    Serial.println(F("======================================"));
```

```
      Serial.println(F("      GAS / SMOKE ALERT"));
      Serial.println(F("======================================="));
    }
}



// ================================================================
//              START FIRE FIGHTING
// ================================================================

void startFireFighting(bool gasDetected)
{
   Serial.println();
   Serial.println(F("======================================="));
   Serial.println(F("       FIRE FIGHTING START"));
   Serial.println(F("======================================="));



   // ============================================================
   // FIRE STATE
   // ============================================================

   fireActive = true;

   fireMode = FIRE_WAIT_FOR_CALL;
```

```
fireTimer = millis();

extinguishCycle = 1;

alignStep = 0;

fireGasDetected = gasDetected;

// LATCH THIS FIRE EVENT.
// No second call/SMS event can be generated until the fire
// has been confirmed extinguished.
fireEventLatched = true;
fireClearConfirmCount = 0;

fireCallStarted = false;

fireCallWaiting = true;



// ============================================================
// IMMEDIATE STOP
// ============================================================

stopMotors();

digitalWrite(PUMP_RELAY, LOW);
```

```
  nozzleServo.write(SERVO_CENTER);

  currentServoAngle = SERVO_CENTER;


  Serial.println(F("[ROVER] STOPPED"));
  Serial.println(F("[FIRE] CALL-FIRST SEQUENCE STARTED"));


  // ============================================================
  // GSM CALL FIRST
  // ============================================================

  // Cancel a queued gas-alert SMS so the emergency call gets
  // priority. The GSM state-machine itself is unchanged.
  smsPending = false;

  requestCall();

  Serial.println(F("[GSM] CALL QUEUED FIRST"));
  Serial.println(F("[FIRE] WAITING FOR CALL TO FINISH"));
}


// ============================================================
```

```
//              UPDATE FIRE ALIGNMENT
// ================================================================


void updateAlignment()
{
  unsigned long now = millis();



  FireState fire = readFireSensors();


  printFireState(fire);



  // ============================================================
  // FIRE LOST
  // ============================================================


  if (!fire.detected)
  {
    Serial.println(F("[ALIGN] FIRE LOST"));


    stopMotors();


    digitalWrite(PUMP_RELAY, LOW);


    fireTimer = now;
```

```
  fireMode = FIRE_RECHECK;

  return;
}


// ============================================================
// FRONT FIRE
// ============================================================

if (fire.front)
{
  Serial.println(F("[ALIGN] FIRE IN FRONT"));
  Serial.println(F("[ALIGN] ROVER CENTERED"));

  // Move slightly forward before extinguishing
  Serial.println(F("[ALIGN] MOVING FORWARD BEFORE SPRAY"));

  moveForward();

  delay(MOVE_FORWARD_BEFORE_SPRAY);

  stopMotors();

  Serial.println(F("[ALIGN] FORWARD MOVEMENT COMPLETE"));
```

```
    Serial.println(F("[ALIGN] STARTING EXTINGUISHMENT"));

    fireTimer = millis();
    fireMode = FIRE_SPRAY_SETTLE;

    return;
}

// ============================================================
// ALIGNMENT TIMER
// ============================================================

if (now - fireTimer < ALIGN_STEP_TIME)
{
    return;
}



// ============================================================
// ALIGNMENT TIMEOUT
// ============================================================

if (alignStep >= MAX_ALIGN_STEPS)
{
    stopMotors();
```

```
    Serial.println(F("[ALIGN] TIMEOUT"));

    fireTimer = now;

    fireMode = FIRE_RECHECK;

    return;
  }



  // ============================================================
  // LEFT FIRE
  // ============================================================

  if (fire.left && !fire.right)
  {
    Serial.println(F("[ALIGN] FIRE LEFT -> TURN LEFT"));

    turnLeft();

    fireTimer = now;

    alignStep++;

    return;
  }
```

```
// ============================================================
// RIGHT FIRE
// ============================================================

if (fire.right && !fire.left)
{
  Serial.println(F("[ALIGN] FIRE RIGHT -> TURN RIGHT"));

  turnRight();

  fireTimer = now;

  alignStep++;

  return;
}



// ============================================================
// BOTH SIDES
// ============================================================

if (fire.left &&
  fire.right &&
```

```
    !fire.front)
{
    Serial.println(F("[ALIGN] FIRE BOTH SIDES"));
    Serial.println(F("[ALIGN] SMALL LEFT CORRECTION"));

    setMotorSpeed(FINE_ALIGN_SPEED);

    M1.run(FORWARD);
    M2.run(FORWARD);
    M3.run(FORWARD);
    M4.run(FORWARD);

    fireTimer = now;

    alignStep++;

    return;
}



// ============================================================
// SAFETY
// ============================================================

stopMotors();
```

```
  fireTimer = now;
}


// ================================================================
//                SPRAY SETTLE
// ================================================================

void updateSpraySettle()
{
  unsigned long now = millis();


  stopMotors();

  digitalWrite(PUMP_RELAY, LOW);


  nozzleServo.write(SERVO_LEFT);

  currentServoAngle = SERVO_LEFT;


  if (now - fireTimer >= SERVO_SETTLE_TIME)
  {
    Serial.println();
```

```
      Serial.println(F("---------------------------------------"));
      Serial.println(F("[SPRAY] START"));
      Serial.println(F("---------------------------------------"));

      Serial.println(F("[SERVO] POSITION = 50 DEG"));

      Serial.println(F("[PUMP] ON"));

      digitalWrite(PUMP_RELAY, HIGH);

      Serial.println(F("[SPRAY] 50 -> 130"));



      fireTimer = now;

      fireMode = FIRE_SPRAY_LEFT;
    }
}



// ================================================================
//                SPRAY 50 -> 130
// ================================================================

void updateSprayLeft()
{
```

```
unsigned long now = millis();



if (now - fireTimer < SWEEP_STEP_DELAY)
{
  return;
}



fireTimer = now;



if (currentServoAngle <= SERVO_RIGHT)
{
  nozzleServo.write(currentServoAngle);

  currentServoAngle++;

  return;
}



Serial.println(F("[SPRAY] 130 -> 50"));

currentServoAngle = SERVO_RIGHT;
```

```
  fireMode = FIRE_SPRAY_RIGHT;
}



// ================================================================
//                SPRAY 130 -> 50
// ================================================================

void updateSprayRight()
{
  unsigned long now = millis();



  if (now - fireTimer < SWEEP_STEP_DELAY)
  {
    return;
  }



  fireTimer = now;



  if (currentServoAngle >= SERVO_LEFT)
  {
    nozzleServo.write(currentServoAngle);
```

```
    currentServoAngle--;

    return;
  }

  // ============================================================
  // COMPLETE
  // ============================================================

  digitalWrite(PUMP_RELAY, LOW);

  Serial.println(F("[PUMP] OFF"));

  nozzleServo.write(SERVO_CENTER);

  currentServoAngle = SERVO_CENTER;

  Serial.println(F("[SERVO] RETURNING TO 90 DEG"));

  Serial.println(F("[SPRAY] COMPLETE"));

  fireTimer = now;
```

```
  fireMode = FIRE_RECHECK;
}



// ================================================================
//                  FIRE RECHECK
// ================================================================

void updateFireRecheck()
{
  unsigned long now = millis();



  if (now - fireTimer < RECHECK_DELAY)
  {
    return;
  }



  FireState fire = readFireSensors();



  Serial.println();
  Serial.println(F("----------------------------------------"));
  Serial.println(F("[FIRE CHECK]"));
  Serial.println(F("----------------------------------------"));
```

```
printFireState(fire);



// ============================================================
// FIRE GONE
// ============================================================

if (!fire.detected)
{
  Serial.println();
  Serial.println(F("======================================="));
  Serial.println(F("       FIRE EXTINGUISHED"));
  Serial.println(F("======================================="));

  digitalWrite(PUMP_RELAY, LOW);

  stopMotors();

  nozzleServo.write(SERVO_CENTER);

  currentServoAngle = SERVO_CENTER;

  fireActive = false;

  fireMode = FIRE_IDLE;
```

```
    extinguishCycle = 0;

    alignStep = 0;

    // Keep the fire event latched for several consecutive
    // clear sensor readings before accepting another fire.
    fireEventLatched = true;
    fireClearConfirmCount = 0;



    Serial.println(F("[ROVER] STOPPED"));
    Serial.println(F("[PUMP] OFF"));
    Serial.println(F("[SERVO] CENTER"));
    Serial.println(F("[SYSTEM] WAITING FOR NEXT FIRE"));

    return;
}



// ============================================================
// FIRE STILL PRESENT
// ============================================================

Serial.println(F("[FIRE] FIRE STILL PRESENT"));
```

```
Serial.print(F("[FIRE] CURRENT CYCLE: "));
Serial.println(extinguishCycle);



// ============================================================
// MAXIMUM CYCLES
// ============================================================

if (extinguishCycle >= MAX_EXTINGUISH_CYCLES)
{
  Serial.println();
  Serial.println(F("========================================"));
  Serial.println(F("      MAXIMUM CYCLES REACHED"));
  Serial.println(F("      FIRE STILL PRESENT"));
  Serial.println(F("========================================"));


  digitalWrite(PUMP_RELAY, LOW);


  stopMotors();


  nozzleServo.write(SERVO_CENTER);


  currentServoAngle = SERVO_CENTER;
```

```
  fireActive = false;

  fireMode = FIRE_IDLE;

  extinguishCycle = 0;

  alignStep = 0;

  // Keep the fire event locked. A new call will not be made
  // unless the flame sensors later confirm the fire is clear.
  fireEventLatched = true;
  fireClearConfirmCount = 0;



  Serial.println(F("[ROVER] STOPPED"));
  Serial.println(F("[PUMP] OFF"));
  Serial.println(F("[SERVO] CENTER"));
  Serial.println(F("[SYSTEM] WAITING"));

  return;
}



// ============================================================
// NEXT CYCLE
// ============================================================
```

```
  extinguishCycle++;

  alignStep = 0;

  Serial.print(F("[FIRE] STARTING CYCLE "));
  Serial.println(extinguishCycle);

  stopMotors();

  digitalWrite(PUMP_RELAY, LOW);

  nozzleServo.write(SERVO_CENTER);

  currentServoAngle = SERVO_CENTER;

  fireTimer = now;

  fireMode = FIRE_ALIGN;
}

// ============================================================
```

```
//              WAIT FOR GSM CALL
// ================================================================

void updateFireCallWait()
{
  unsigned long now = millis();

  // Keep the rover completely stationary until the emergency call
  // has actually entered the GSM state machine.
  stopMotors();

  digitalWrite(PUMP_RELAY, LOW);

  nozzleServo.write(SERVO_CENTER);

  currentServoAngle = SERVO_CENTER;



  // ----------------------------------------------------------
  // CALL HAS STARTED
  // ----------------------------------------------------------

  if (gsmState == GSM_CALL_DIAL_WAIT ||
    gsmState == GSM_CALL_ACTIVE)
  {
    fireCallStarted = true;
```

```
}



// ------------------------------------------------------------
// CALL FINISHED
// ------------------------------------------------------------

if (fireCallStarted &&
   gsmState == GSM_IDLE &&
   !callPending)
{
   Serial.println(F("[GSM] CALL FINISHED"));
   Serial.println(F("[FIRE] STARTING EXTINGUISHMENT"));

   fireCallStarted = false;
   fireCallWaiting = false;



   // Queue the fire SMS only after the call is complete.
   // This uses the exact pasted GSM request/update logic.
   if (fireGasDetected)
   {
      requestSMS(
         "FIRE ALERT! Fire detected. GAS/SMOKE also detected. Robot is extinguishing."
      );
   }
```

```
      else
      {
        requestSMS(
          "FIRE ALERT! Fire detected. No gas/smoke detected. Robot is extinguishing."
        );
      }


      fireTimer = now;

      fireMode = FIRE_ALIGN;

      alignStep = 0;

      return;
    }
}


// ================================================================
//              UPDATE FIRE CONTROLLER
// ================================================================

void updateFireController()
{
```

```
if (!fireActive)
{
   return;
}


switch (fireMode)
{
   case FIRE_WAIT_FOR_CALL:

      updateFireCallWait();

      break;


   case FIRE_ALIGN:

      updateAlignment();

      break;


   case FIRE_SPRAY_SETTLE:

      updateSpraySettle();
```

```
    break;

case FIRE_SPRAY_LEFT:
    updateSprayLeft();
    break;

case FIRE_SPRAY_RIGHT:
    updateSprayRight();
    break;

case FIRE_RECHECK:
    updateFireRecheck();
    break;

case FIRE_IDLE:
```

```
      default:

        stopMotors();

        break;
    }
}



// ================================================================
//             UPDATE FIRE EVENT LATCH
// ================================================================

void updateFireEventLatch()
{
    // A confirmed fire event is already active.
    // Do not clear the latch while the fire controller is working.
    if (fireActive)
    {
      fireClearConfirmCount = 0;
      return;
    }

    // No active fire-fighting process, but the previous fire event
    // is still latched. Require several consecutive clear readings
    // before allowing another fire event.
```

```
  if (fireEventLatched)
  {
    FireState fire = readFireSensors();

    if (!fire.detected)
    {
      fireClearConfirmCount++;

      if (fireClearConfirmCount >= FIRE_CLEAR_CONFIRM_COUNT)
      {
        fireEventLatched = false;
        fireClearConfirmCount = 0;

        Serial.println(F("[FIRE LATCH] FIRE EVENT CLEARED"));
        Serial.println(F("[SYSTEM] READY FOR NEXT FIRE"));
      }
    }
    else
    {
      fireClearConfirmCount = 0;
    }
  }
}


// ============================================================
```

```
//                  CHECK FOR FIRE
// ================================================================


void checkForFire()
{
  if (fireActive)
  {
    return;
  }

  // IMPORTANT:
  // Once a fire has been detected, do not create another
  // call/SMS event until the previous fire event is cleared.
  if (fireEventLatched)
  {
    return;
  }



  FireState fire = readFireSensors();



  if (!fire.detected)
  {
    // No fire = absolutely stationary
    stopMotors();
```

```
    return;
}



Serial.println();
Serial.println(F("========================================"));
Serial.println(F("      !!! POSSIBLE FIRE !!!"));
Serial.println(F("========================================"));



printFireState(fire);



if (!confirmFire())
{
    Serial.println(F("[FIRE] NOT CONFIRMED"));


    stopMotors();


    return;
}



Serial.println();
Serial.println(F("========================================"));
```

```
  Serial.println(F("        !!! FIRE CONFIRMED !!!"));
  Serial.println(F("======================================="));



  bool gasDetected =
    (digitalRead(GAS_SENSOR) == LOW);



  Serial.print(F("[GAS] "));

  if (gasDetected)
    Serial.println(F("GAS / SMOKE DETECTED"));
  else
    Serial.println(F("NORMAL"));



  startFireFighting(gasDetected);
}



// ============================================================
//                    SETUP
// ============================================================

void setup()
{
```

```
// ============================================================
// SERIAL
// ============================================================

Serial.begin(115200);

sim800L.begin(9600);



// ============================================================
// FLAME SENSORS
// ============================================================

pinMode(FLAME_LEFT, INPUT);
pinMode(FLAME_FRONT, INPUT);
pinMode(FLAME_RIGHT, INPUT);



// ============================================================
// GAS
// ============================================================

pinMode(GAS_SENSOR, INPUT);



// ============================================================
```

```
// PUMP
// ============================================================

pinMode(PUMP_RELAY, OUTPUT);

digitalWrite(PUMP_RELAY, LOW);



// ============================================================
// SERVO
// ============================================================

nozzleServo.attach(SERVO_PIN);

nozzleServo.write(SERVO_CENTER);

currentServoAngle = SERVO_CENTER;

delay(500);



// ============================================================
// MOTOR
// ============================================================

stopMotors();
```

```
// ============================================================
// SIM800L
// ============================================================

delay(3000);

clearSIM800();



sim800L.println(F("AT"));

delay(1000);

processSIM800Input();



sim800L.println(F("ATE0"));

delay(500);

processSIM800Input();



sim800L.println(F("AT+CMGF=1"));
```

```
delay(500);

processSIM800Input();


clearSIM800();


// ============================================================
// INITIAL STATE
// ============================================================

fireActive = false;

fireMode = FIRE_IDLE;

extinguishCycle = 0;

alignStep = 0;

digitalWrite(PUMP_RELAY, LOW);

nozzleServo.write(SERVO_CENTER);

currentServoAngle = SERVO_CENTER;
```

```
stopMotors();



// ============================================================
// STARTUP OUTPUT
// ============================================================

Serial.println();
Serial.println(F("========================================"));
Serial.println(F("       FIRE FIGHTING ROVER"));
Serial.println(F("========================================"));

Serial.println();

Serial.println(F("CONTROLLER : Arduino UNO"));
Serial.println(F("DRIVER     : L293D MOTOR SHIELD V1"));

Serial.println();

Serial.println(F("MOTOR MAPPING"));
Serial.println(F("---------------------------------------"));
Serial.println(F("M1 = LEFT FRONT"));
Serial.println(F("M2 = LEFT REAR"));
Serial.println(F("M3 = RIGHT FRONT"));
Serial.println(F("M4 = RIGHT REAR"));
```

```
Serial.println();

Serial.println(F("MOVEMENT LOGIC"));
Serial.println(F("----------------------------------------"));
Serial.println(F("FORWARD  = M1/M2 BACKWARD + M3/M4 FORWARD"));
Serial.println(F("BACKWARD = M1/M2 FORWARD + M3/M4 BACKWARD"));
Serial.println(F("LEFT     = ALL MOTORS FORWARD"));
Serial.println(F("RIGHT    = ALL MOTORS BACKWARD"));

Serial.println();

Serial.println(F("SENSOR PINS"));
Serial.println(F("----------------------------------------"));
Serial.println(F("A0 = LEFT FLAME"));
Serial.println(F("A1 = FRONT FLAME"));
Serial.println(F("A2 = RIGHT FLAME"));
Serial.println(F("A3 = MQ-2"));

Serial.println();

Serial.println(F("COMMUNICATION / OUTPUT"));
Serial.println(F("----------------------------------------"));
Serial.println(F("A4 = SIM800L TX"));
Serial.println(F("D2 = SIM800L RX"));
Serial.println(F("A5 = PUMP"));
```

```
Serial.println(F("D9 = SERVO"));


Serial.println();


Serial.println(F("SERVO"));
Serial.println(F("----------------------------------------"));
Serial.println(F("LEFT   = 50 DEG"));
Serial.println(F("CENTER = 90 DEG"));
Serial.println(F("RIGHT  = 130 DEG"));


Serial.println();


Serial.println(F("AUTONOMOUS MOVEMENT"));
Serial.println(F("----------------------------------------"));
Serial.println(F("NO AUTOMATIC SEARCH"));
Serial.println(F("INITIAL MOVEMENT = STOP"));
Serial.println(F("NO FIRE = STOP"));


Serial.println();


Serial.println(F("FIRE ALIGNMENT"));
Serial.println(F("----------------------------------------"));
Serial.println(F("LEFT FIRE  -> LEFT"));
Serial.println(F("FRONT FIRE -> STOP"));
Serial.println(F("RIGHT FIRE -> RIGHT"));
```

```
Serial.println();

Serial.println(F("EXTINGUISHMENT"));
Serial.println(F("---------------------------------------"));
Serial.println(F("PUMP = ON"));
Serial.println(F("SERVO = 50 -> 130 -> 50"));
Serial.println(F("MAX CYCLES = 5"));

Serial.println();

Serial.println(F("GSM"));
Serial.println(F("---------------------------------------"));
Serial.println(F("CALL FIRST -> EXTINGUISH -> SMS"));

Serial.println();

Serial.println(F("======================================="));
Serial.println(F("          SYSTEM READY"));
Serial.println(F("======================================="));

Serial.println();
Serial.println(F("[ROVER] STOPPED"));
Serial.println(F("[PUMP] OFF"));
Serial.println(F("[SERVO] 90 DEG"));
Serial.println(F("[SYSTEM] WAITING FOR FIRE"));
Serial.println();
```

```
}


// ============================================================
//                      LOOP
// ============================================================

void loop()
{
  // ============================================================
  // 1. GSM
  // ============================================================

  updateGSM();



  // ============================================================
  // 2. FIRE CONTROLLER
  // ============================================================

  updateFireController();



  // ============================================================
  // 3. FIRE EVENT LATCH
  // ============================================================
```

```
// Keep one fire event active until the fire is confirmed clear.
// This prevents repeated calls/SMS from repeated detections.
updateFireEventLatch();



// ============================================================
// 4. FIRE DETECTION
// ============================================================

// Fire detection gets priority over a gas-only SMS so that
// the emergency call can be queued first.
checkForFire();



// ============================================================
// 5. GAS SENSOR
// ============================================================

bool gasDetected =
  (digitalRead(GAS_SENSOR) == LOW);



if (!fireActive && gasDetected)
{
  handleGasAlert();
```

```
  }
  else if (!gasDetected)
  {
    gasAlertSent = false;
  }



  // ============================================================
  // 6. FINAL SAFETY CONDITION
  // ============================================================

  if (!fireActive &&
    fireMode == FIRE_IDLE)
  {
    stopMotors();


    digitalWrite(PUMP_RELAY, LOW);
  }
}
```